# Issue Tracking

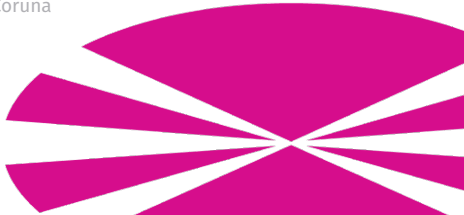
## Ferramentas de Desenvolvemento

Javier Parapar López

Grao en Enxeñaría Informática · Facultade de Informática · Universidade da Coruña

Curso 2026/2027

GEI GRADO EN ENXEÑARÍA INFORMÁTICA



# Where we are going

1. Why a tracker
2. Redmine
3. Issues
4. Time
5. Planning
6. Documents
7. Code
8. Working this way

SECTION 1

# Why a tracker

1

# The shared memory of a team

An issue tracking system is the single place that records **what has to be done, who is doing it, and what happened**. Everything else — chat, email, the conversation after class — forgets.

## Questions it answers

Who is responsible for this? · What is left before the next version? · Has anybody seen this bug before? · Why was it decided like this? · What did we actually do last week?

# The shared memory of a team

An issue tracking system is the single place that records **what has to be done, who is doing it, and what happened**. Everything else — chat, email, the conversation after class — forgets.

## Questions it answers

Who is responsible for this? · What is left before the next version? · Has anybody seen this bug before? · Why was it decided like this? · What did we actually do last week?

## What it costs not to have one

Six people in a group of 6 keep the state of the project in **six heads and a chat group**. Two of you fix the same bug; a third is blocked on something nobody knows about; and at the end nobody can say what was done, or by whom.

# The shared memory of a team

An issue tracking system is the single place that records **what has to be done, who is doing it, and what happened**. Everything else — chat, email, the conversation after class — forgets.

## Questions it answers

Who is responsible for this? · What is left before the next version? · Has anybody seen this bug before? · Why was it decided like this? · What did we actually do last week?

## What it costs not to have one

Six people in a group of 6 keep the state of the project in **six heads and a chat group**. Two of you fix the same bug; a third is blocked on something nobody knows about; and at the end nobody can say what was done, or by whom.

A ticket — an **issue** in Redmine — is one entry in that memory: a description, a type, a state, an owner, a history of everything that happened to it, and links to the code that changed because of it.

# The shared memory of a team

An issue tracking system is the single place that records **what has to be done, who is doing it, and what happened**. Everything else — chat, email, the conversation after class — forgets.

## Questions it answers

Who is responsible for this? · What is left before the next version? · Has anybody seen this bug before? · Why was it decided like this? · What did we actually do last week?

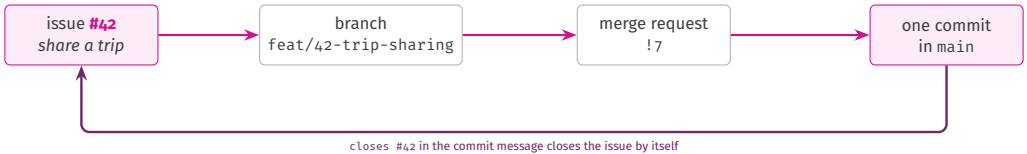
## What it costs not to have one

Six people in a group of 6 keep the state of the project in **six heads and a chat group**. Two of you fix the same bug; a third is blocked on something nobody knows about; and at the end nobody can say what was done, or by whom.

A ticket — an **issue** in Redmine — is one entry in that memory: a description, a type, a state, an owner, a history of everything that happened to it, and links to the code that changed because of it.

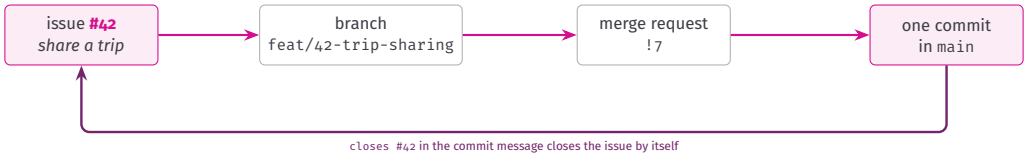
***And it is not only for bugs. Any unit of work is an issue: a feature, a refactor, a document to write, a decision to take.***

# The issue is the unit of work




This is the same chain you agreed to in *Lesson 2*, read from the other end. The issue number is what holds it together: it is in the **branch name**, in the **commit message**, in the **merge request**, and it is what lets you get from a line of code back to the reason it exists.

# The issue is the unit of work




This is the same chain you agreed to in *Lesson 2*, read from the other end. The issue number is what holds it together: it is in the **branch name**, in the **commit message**, in the **merge request**, and it is what lets you get from a line of code back to the reason it exists.

***The test of whether a team really works this way: pick any line in `main`, and try to reach the discussion that produced it in two clicks. If you can, everything in this lesson is being used properly.***

# What makes a tracker work — or not

**It has to be easy**

- Filing an issue must be **simple**, for everybody: a tracker people avoid is an empty tracker
- The information must appear **where the decision is taken** — in the editor, in the merge request — not only in a browser tab
- Agree the **vocabulary**: if *resolved* means something different to each of you, the board means nothing

**And it has to be honest**

- Capture enough to **report**: dates, author, type, version, time
- Defect reports must be **reproducible**, with a screenshot when words are ambiguous
- **Search before you file**: duplicates split the discussion in two and neither half has the answer
- The states must **match how you actually work**, and the whole thing must be **connected to the repository**

# What makes a tracker work — or not

**It has to be easy**

- Filing an issue must be **simple**, for everybody: a tracker people avoid is an empty tracker
- The information must appear **where the decision is taken** — in the editor, in the merge request — not only in a browser tab
- Agree the **vocabulary**: if *resolved* means something different to each of you, the board means nothing

**And it has to be honest**

- Capture enough to **report**: dates, author, type, version, time
- Defect reports must be **reproducible**, with a screenshot when words are ambiguous
- **Search before you file**: duplicates split the discussion in two and neither half has the answer
- The states must **match how you actually work**, and the whole thing must be **connected to the repository**

***The common failure in student projects is not choosing the wrong tool. It is filing the issue after doing the work, which turns the tracker into a diary nobody reads instead of a plan somebody follows.***

# Everyone has one, and they all look alike

Bugzilla, Trac, **Jira**, **Redmine**, GitLab issues, GitHub issues, Linear, Azure Boards… The names differ; the model does not:

| HERE | ELSEWHERE | WHAT IT IS |
|---|---|---|
| Issue | ticket, work item, story, card | one unit of work |
| Tracker | issue type, work item type | bug, feature, support… |
| Status | state, column on a board | where it is in the process |
| Target version | milestone, sprint, fix version | when it should be done |
| Assignee | owner | who is doing it |
| Category | component, label, epic | which part of the system |

# Everyone has one, and they all look alike

Bugzilla, Trac, **Jira**, **Redmine**, GitLab issues, GitHub issues, Linear, Azure Boards… The names differ; the model does not:

| HERE | ELSEWHERE | WHAT IT IS |
|---|---|---|
| Issue | ticket, work item, story, card | one unit of work |
| Tracker | issue type, work item type | bug, feature, support… |
| Status | state, column on a board | where it is in the process |
| Target version | milestone, sprint, fix version | when it should be done |
| Assignee | owner | who is doing it |
| Category | component, label, epic | which part of the system |

We use **Redmine** because it is what the faculty runs, it is free software, and it does time tracking properly — which matters for the individual follow-up of this course. What you learn transfers: in your first job it will be Jira, and you will recognise every field.

SECTION 2

# Redmine

2

# Redmine

- A web-based **project management** tool, free software, one instance serving many projects
- Its core element is the `issue`: a specific, actionable task of **any** kind, not only programming — which is how a large project becomes tractable, broken into pieces small enough to finish
- And around those issues, the rest of the machinery:

| MODULE | FOR |
|---|---|
| Activity | what happened lately, by everybody |
| Roadmap | what is left before each version |
| Wiki | decisions, designs, how-tos |
| News / Forums | announcements / discussion |
| Documents / Files | things that are not source code |
| Time tracking | hours spent, per issue and per person |
| Repository | the code, linked to the issues |



SECTION 3

# Issues

3

# The fields of an issue, and what they are for

| FIELD | THE QUESTION IT ANSWERS |
|---|---|
| **Tracker** | what kind of work is this? *Bug*, *Feature*, *Support* |
| **Subject** | what is it, in one line somebody can scan |
| **Description** | everything the next person needs in order to start |
| **Status** | where is it in the process? |
| **Priority** | *Low, Normal, High, Urgent, Immediate* — and if everything is urgent, nothing is |
| **Assignee** | who is doing it **now** (one person, not a group) |
| **Target version** | which iteration it belongs to |
| **Category** | which part of the system it touches |
| **Parent task** | is it a piece of something bigger? |
| **Estimated time** / **% done** | how big we thought it was / how far it has got |
| **Watchers** | who else wants to hear about it |

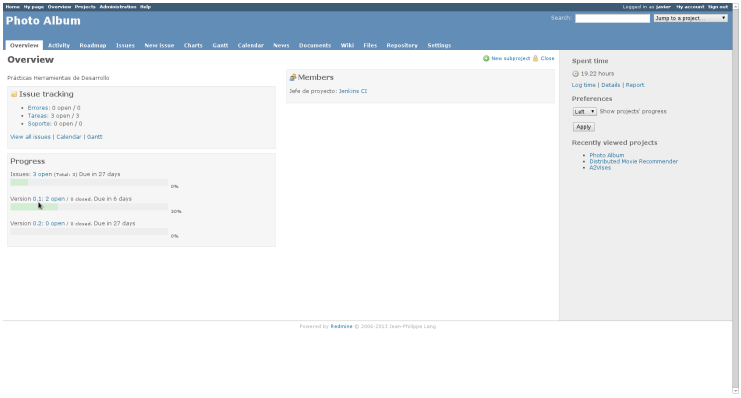
# The fields of an issue, and what they are for

| FIELD | THE QUESTION IT ANSWERS |
|---|---|
| **Tracker** | what kind of work is this? *Bug*, *Feature*, *Support* |
| **Subject** | what is it, in one line somebody can scan |
| **Description** | everything the next person needs in order to start |
| **Status** | where is it in the process? |
| **Priority** | *Low, Normal, High, Urgent, Immediate* — and if everything is urgent, nothing is |
| **Assignee** | who is doing it **now** (one person, not a group) |
| **Target version** | which iteration it belongs to |
| **Category** | which part of the system it touches |
| **Parent task** | is it a piece of something bigger? |
| **Estimated time** / **% done** | how big we thought it was / how far it has got |
| **Watchers** | who else wants to hear about it |

***Two of them are a team agreement rather than data: priority, meaningless if everything is high, and assignee, which must be one person — an issue assigned to everybody is assigned to nobody.***

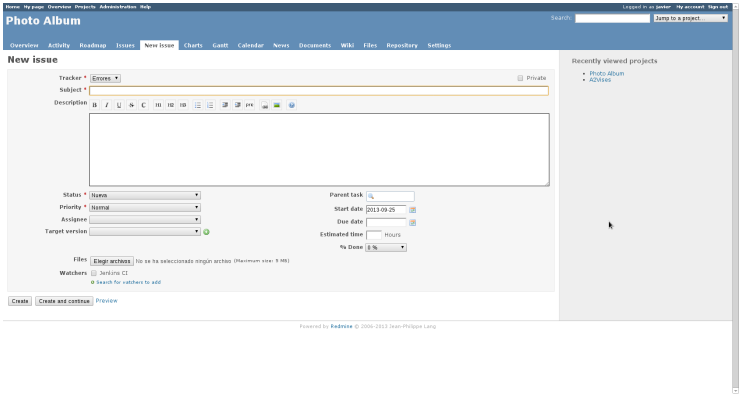
# Creating an issue

The project overview: where the project stands, at a glance.



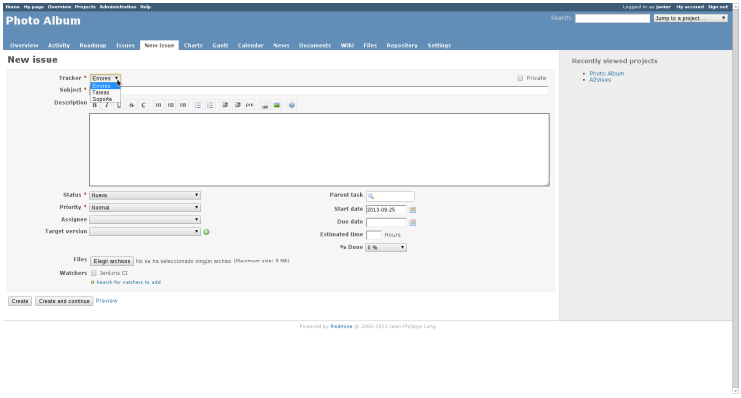
# Creating an issue

**New issue** — if your role in the project allows it.



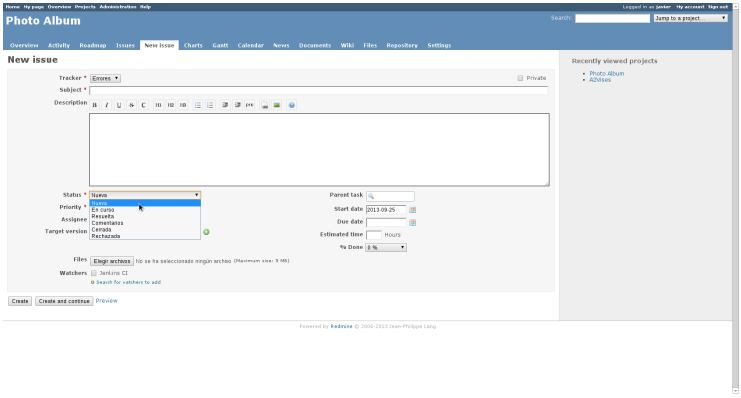
# Creating an issue

**Tracker**: the kind of work. The defaults are Bug, Feature and Support.



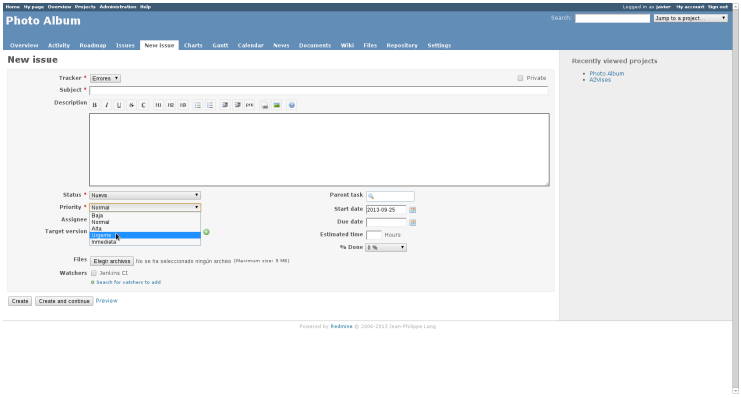
# Creating an issue

**Status**: New, In Progress, Resolved, Feedback, Closed, Rejected.



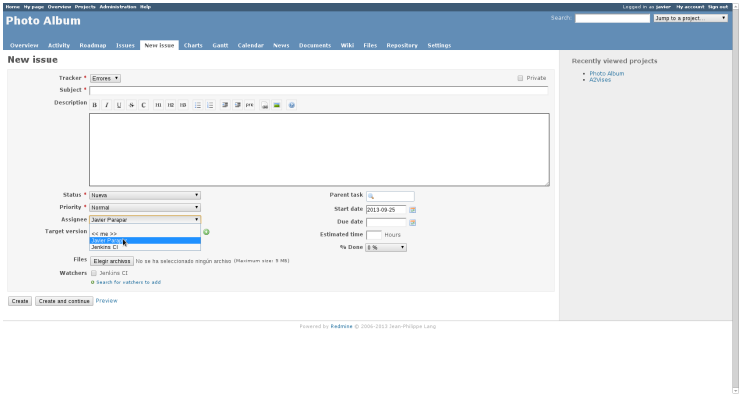
# Creating an issue

**Priority**: Low, Normal, High, Urgent, Immediate.



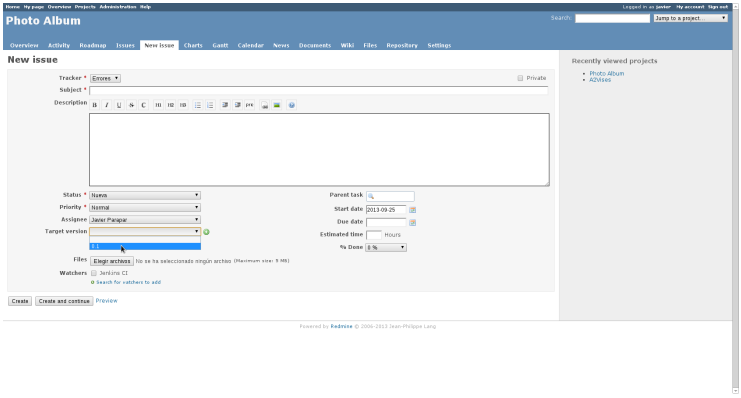
# Creating an issue

**Assignee**: the one person responsible for it right now.



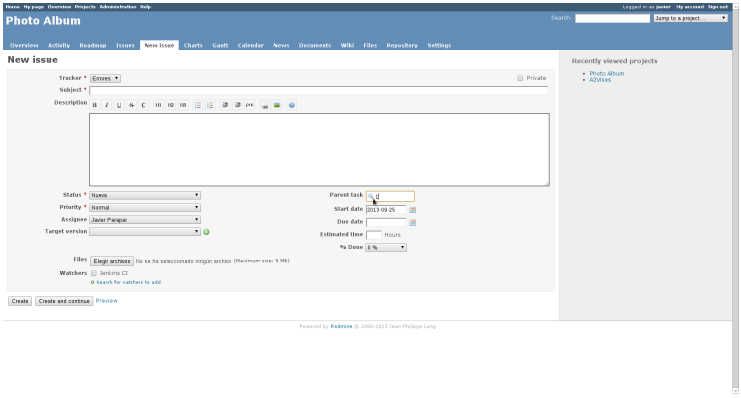
# Creating an issue

**Target version**: the iteration this has to be finished for.



# Creating an issue

**Parent task**: when this is one piece of something bigger.



# How to write an issue somebody can act on

## A bug report

**Subject**: the symptom in one line — *"Saving a trip with no dates returns a 500"*

**Steps to reproduce**: numbered, from a known starting point
**Expected** vs **actual** result
**Environment**: version, branch, browser, data
**Evidence**: the stack trace, the screenshot, the log

# How to write an issue somebody can act on

## A bug report

**Subject**: the symptom in one line — *"Saving a trip with no dates returns a 500"*

**Steps to reproduce**: numbered, from a known starting point
**Expected** vs **actual** result
**Environment**: version, branch, browser, data
**Evidence**: the stack trace, the screenshot, the log

## A task

**Subject**: the outcome, not the activity — *"A trip can be shared with another user"*, not *"work on sharing"*

**Description**: what it must do, what is out of scope, and **how we will know it is done** — the acceptance criteria

Small enough to finish in a day or two. If not, it is a parent task.

# How to write an issue somebody can act on

## A bug report

**Subject**: the symptom in one line — *"Saving a trip with no dates returns a 500"*

**Steps to reproduce**: numbered, from a known starting point
**Expected** vs **actual** result
**Environment**: version, branch, browser, data
**Evidence**: the stack trace, the screenshot, the log

## A task

**Subject**: the outcome, not the activity — *"A trip can be shared with another user"*, not *"work on sharing"*

**Description**: what it must do, what is out of scope, and **how we will know it is done** — the acceptance criteria

Small enough to finish in a day or two. If not, it is a parent task.

The rule behind both: an issue is written **for somebody else** — or for you in three weeks, which is the same thing. If the only person who can act on it is the person who wrote it, it is a note, not an issue.

# How to write an issue somebody can act on

## A bug report

**Subject**: the symptom in one line — *"Saving a trip with no dates returns a 500"*

**Steps to reproduce**: numbered, from a known starting point
**Expected** vs **actual** result
**Environment**: version, branch, browser, data
**Evidence**: the stack trace, the screenshot, the log

## A task

**Subject**: the outcome, not the activity — *"A trip can be shared with another user"*, not *"work on sharing"*

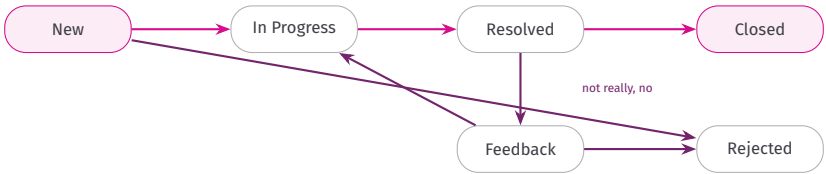
**Description**: what it must do, what is out of scope, and **how we will know it is done** — the acceptance criteria

Small enough to finish in a day or two. If not, it is a parent task.

The rule behind both: an issue is written **for somebody else** — or for you in three weeks, which is the same thing. If the only person who can act on it is the person who wrote it, it is a note, not an issue.

***And search before you file. A duplicate splits the conversation in two, and the half with the answer is never the half you are reading.***

# Statuses are a state machine

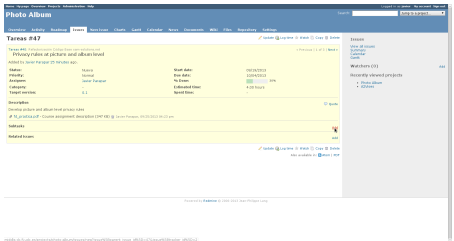



- Redmine defines the allowed transitions per **role** and per **tracker**: a developer may resolve, a manager may close, and not every arrow exists for everybody
- **Resolved is not closed.** Resolved means *I think I am done*; closed means *somebody else agrees* — which is the review of *Lesson 2*, expressed as a state
- A status nobody can leave, or that everybody skips, is not a tracker problem: it is a **process problem** your tracker has made visible

# Issues that depend on issues

Two different things. A subtask **decomposes**: the parent is not finished until its children are. A relation **informs** — *blocks*, *precedes*, *duplicates* — nobody is inside anybody.

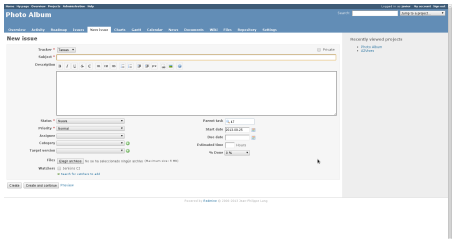
**Related issues** and **Subtasks** are added from the issue itself.



# Issues that depend on issues

Two different things. A subtask **decomposes**: the parent is not finished until its children are. A relation **informs** — *blocks*, *precedes*, *duplicates* — nobody is inside anybody.

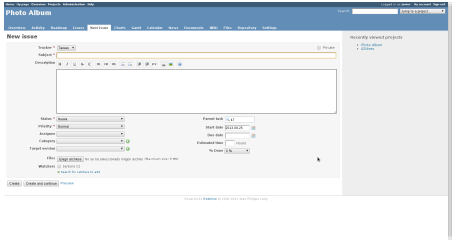
A subtask is just a new issue with the **parent task** filled in.



# Issues that depend on issues

Two different things. A subtask **decomposes**: the parent is not finished until its children are. A relation **informs** — *blocks*, *precedes*, *duplicates* — nobody is inside anybody.

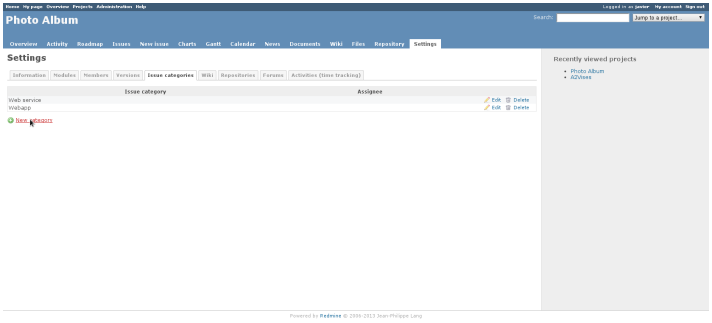
A subtask is just a new issue with the **parent task** filled in.




***Use subtasks when a task was too big — which is the same rule as Lesson 2: if the branch would live more than two days, the issue wanted splitting. And use blocks honestly: a blocked issue that nobody marks as blocked looks exactly like an issue nobody is working on.***

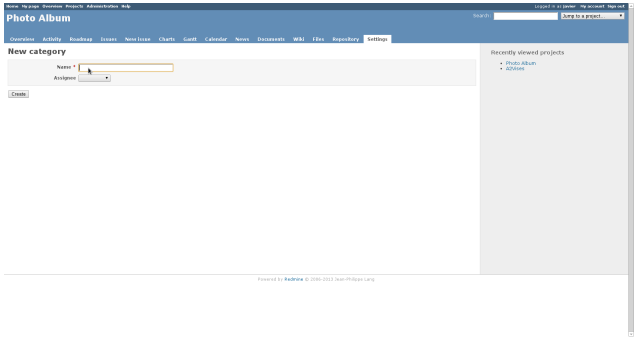
# Categories, people, and who gets the email

**Issue categories** are the parts of *your* system: chosen by you, in *Settings*.



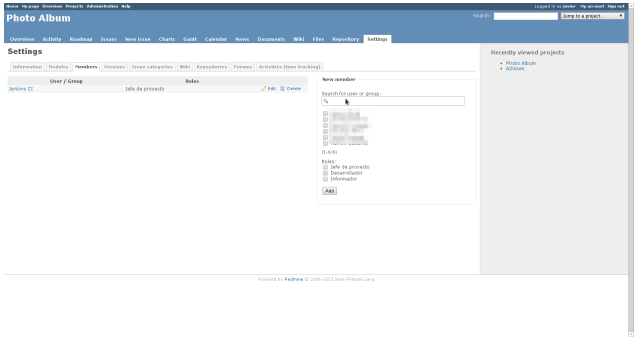
# Categories, people, and who gets the email

Each category can have a **default assignee** — whoever owns that part.



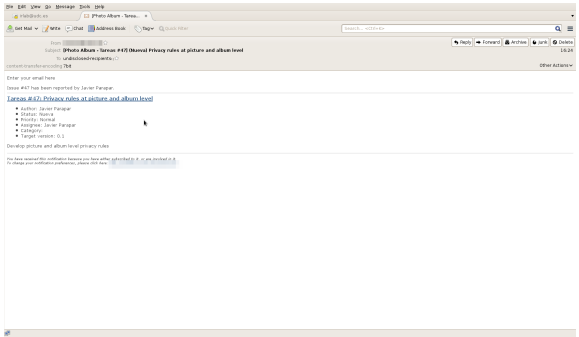
# Categories, people, and who gets the email

**Members** and **roles**: what each person is allowed to do in the project.



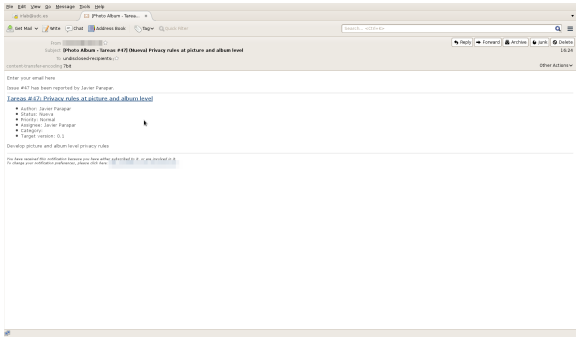
# Categories, people, and who gets the email

And the consequence: the people involved are **notified** without anybody telling them.



# Categories, people, and who gets the email

And the consequence: the people involved are **notified** without anybody telling them.




***Four or five categories are plenty — `frontend`, `api`, `database`, `build`. Twenty categories is a taxonomy nobody maintains.***

SECTION 4

# Time

4

# Why we ask you to record your hours

**What it is for**

- To compare **estimated** against **spent**, and discover that you are systematically out by a factor of two — everybody is, at first
- To make the **distribution of work** visible: six people, one project, and a report that says who did what
- To know what a feature **cost**, which is the only way to price the next one

# Why we ask you to record your hours

**What it is for**

- To compare **estimated** against **spent**, and discover that you are systematically out by a factor of two — everybody is, at first
- To make the **distribution of work** visible: six people, one project, and a report that says who did what
- To know what a feature **cost**, which is the only way to price the next one

**What it is not for**

- It is not a competition. More hours on an issue is not more merit; it usually means the issue was badly cut
- It is not filled in at the end. Hours invented in December are **noise**, and they are obvious

# Why we ask you to record your hours

**What it is for**

- To compare **estimated** against **spent**, and discover that you are systematically out by a factor of two — everybody is, at first
- To make the **distribution of work** visible: six people, one project, and a report that says who did what
- To know what a feature **cost**, which is the only way to price the next one

**What it is not for**

- It is not a competition. More hours on an issue is not more merit; it usually means the issue was badly cut
- It is not filled in at the end. Hours invented in December are **noise**, and they are obvious

Every entry has an **activity**: design, development, analysis, testing, deployment, management… That is what turns a number of hours into a sentence — *"we spent more time on the build than on the feature"* — which is exactly the kind of thing this subject is about.

# Why we ask you to record your hours

**What it is for**

- To compare **estimated** against **spent**, and discover that you are systematically out by a factor of two — everybody is, at first
- To make the **distribution of work** visible: six people, one project, and a report that says who did what
- To know what a feature **cost**, which is the only way to price the next one

**What it is not for**

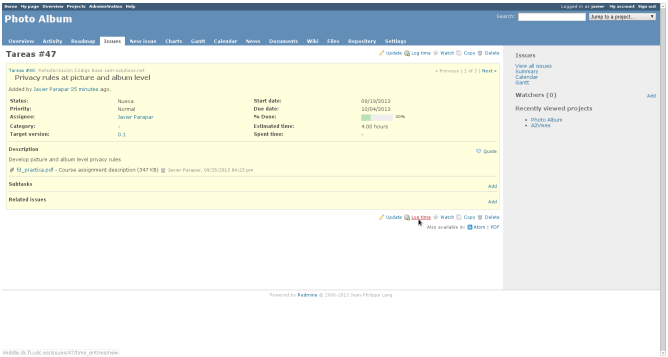
- It is not a competition. More hours on an issue is not more merit; it usually means the issue was badly cut
- It is not filled in at the end. Hours invented in December are **noise**, and they are obvious

Every entry has an **activity**: design, development, analysis, testing, deployment, management… That is what turns a number of hours into a sentence — *"we spent more time on the build than on the feature"* — which is exactly the kind of thing this subject is about.

***In this course the time log is part of the individual follow-up: it is how the work of each member of the team becomes visible, and it is read.***

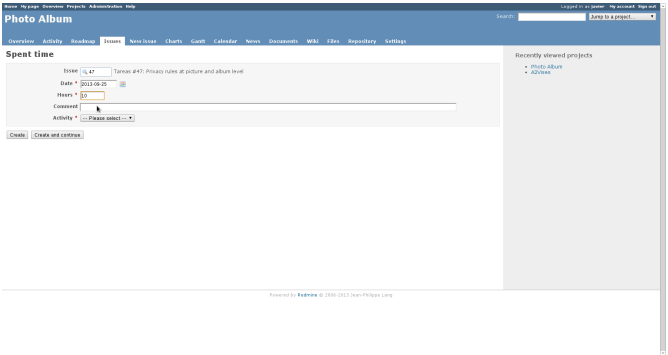
# Logging time on an issue

From the issue: **Log time**.



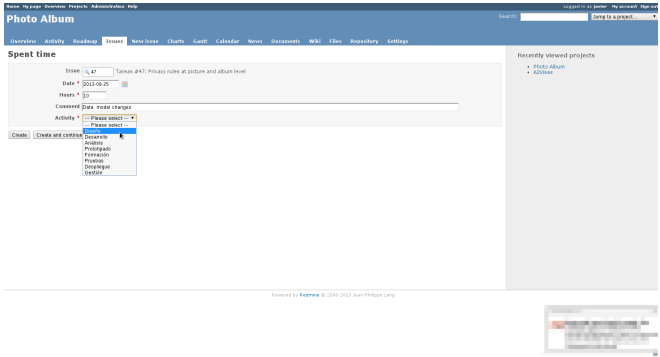
# Logging time on an issue

Date, **hours**, and a comment saying what those hours were.



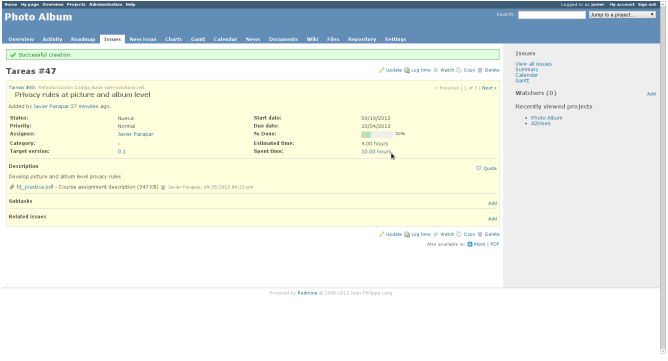
# Logging time on an issue

And the **activity**, from the list the project defines.



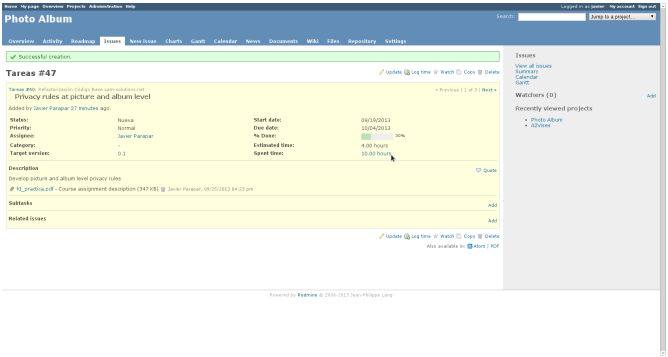
# Logging time on an issue

The issue now carries **spent time** next to the estimate.



# Logging time on an issue

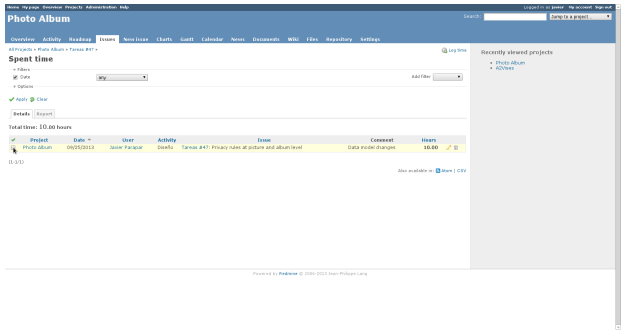
The issue now carries **spent time** next to the estimate.




***Do it the same day. Two days later you are not recording time, you are estimating the past.***

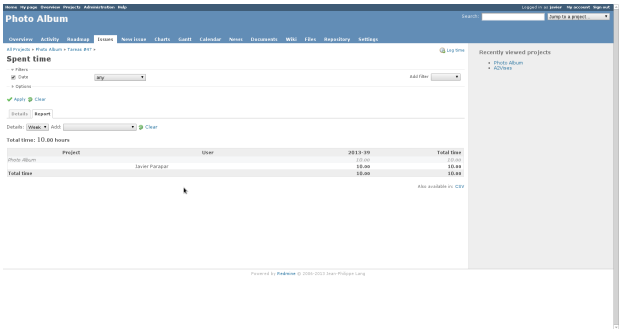
# And reading the hours back

**Details**: every entry, filterable — by person, by date, by activity.



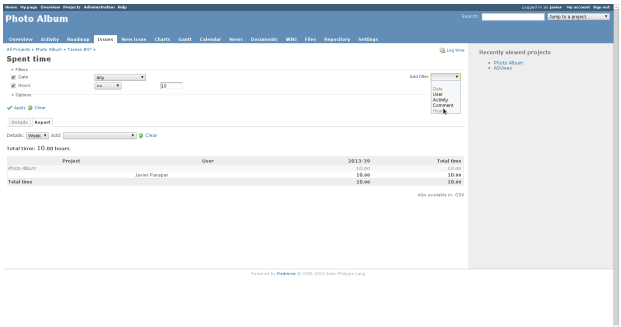
# And reading the hours back

**Report**: the same data aggregated, by week, by member, by project.



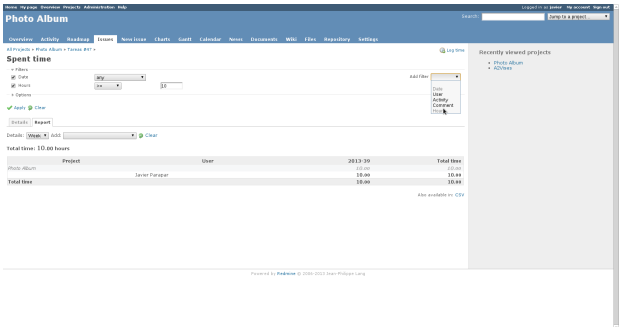
# And reading the hours back

Any criterion can be added as a column, and all of it exports to CSV.



# And reading the hours back

Any criterion can be added as a column, and all of it exports to CSV.




***This report is the one we look at when we ask how the team organised itself. A flat line for one member and a wall for another is not a mark against the wall: it is a question for the team.***

SECTION 5

# Planning

5

# A version is an iteration is a tag




one of the four iterations of the assignment

The **target version** field is what makes planning possible: it turns a pile of issues into *“these are the ones due on the 4th”*. The roadmap is nothing more than that pile grouped by version, with what is done and what is not.

# A version is an iteration is a tag




one of the four iterations of the assignment

The **target version** field is what makes planning possible: it turns a pile of issues into *"these are the ones due on the 4th"*. The roadmap is nothing more than that pile grouped by version, with what is done and what is not.

In this course there are **four iterations**, so four versions, and each one is closed with the annotated tag of *Lesson 2*: `v1.0.0`, `v2.0.0`, `v3.0.0`, `v4.0.0`. The tracker says what should be in it; the tag says what was.

# A version is an iteration is a tag




one of the four iterations of the assignment

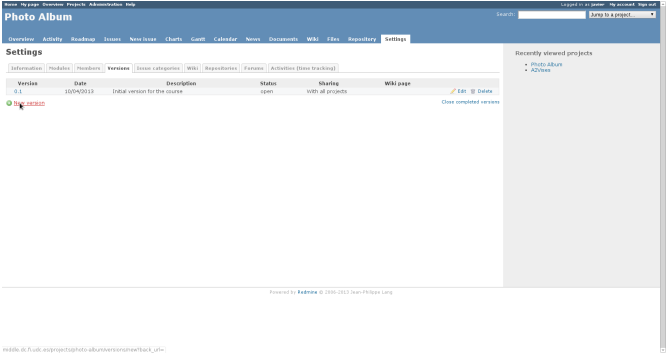
The **target version** field is what makes planning possible: it turns a pile of issues into *"these are the ones due on the 4th"*. The roadmap is nothing more than that pile grouped by version, with what is done and what is not.

In this course there are **four iterations**, so four versions, and each one is closed with the annotated tag of *Lesson 2*: `v1.0.0`, `v2.0.0`, `v3.0.0`, `v4.0.0`. The tracker says what should be in it; the tag says what was.

***An issue with no target version is an issue nobody has promised to anyone.***

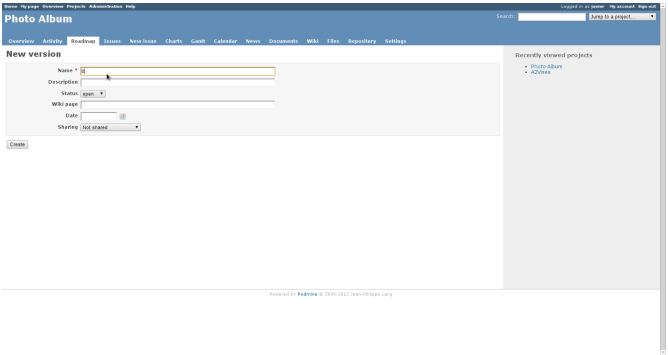
# Creating the versions

*Settings* → **Versions**: the list, and what is still open.



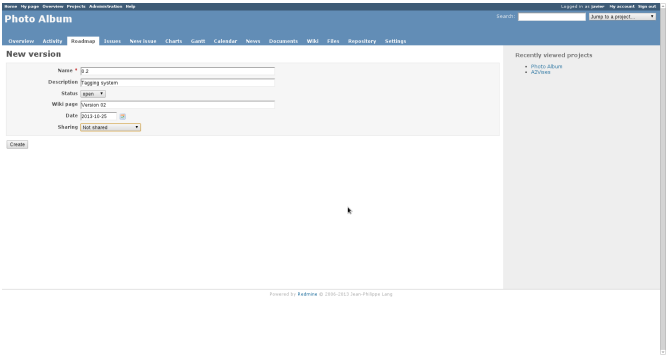
# Creating the versions

A version needs little: a **name**, a **date**, and a wiki page if you want one.



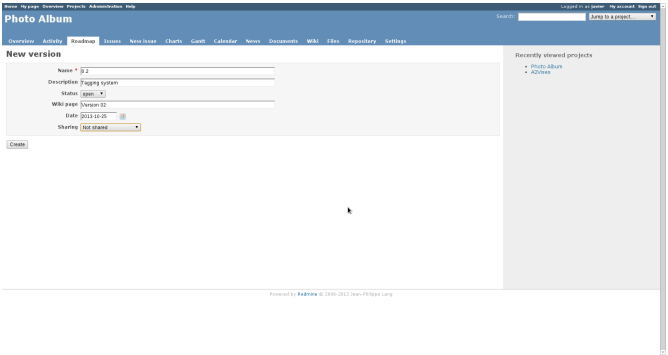
# Creating the versions

One per iteration, created **at the start of the course**, not the night before.



# Creating the versions

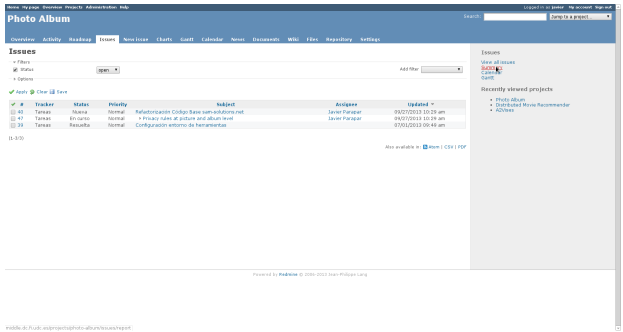
One per iteration, created **at the start of the course**, not the night before.




***The due date is what makes the roadmap and the Gantt say anything. A version with no date is a label.***

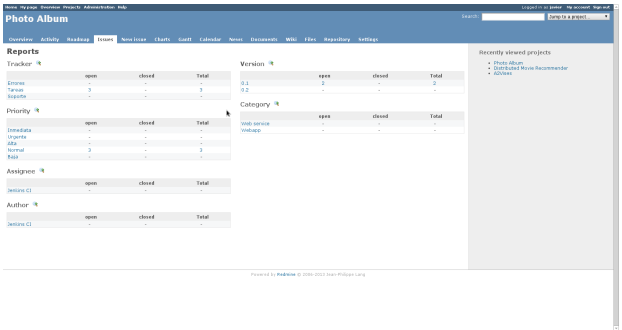
# The same issues, seen four ways

The **list**: filters on any field, saved as a custom query, exported.



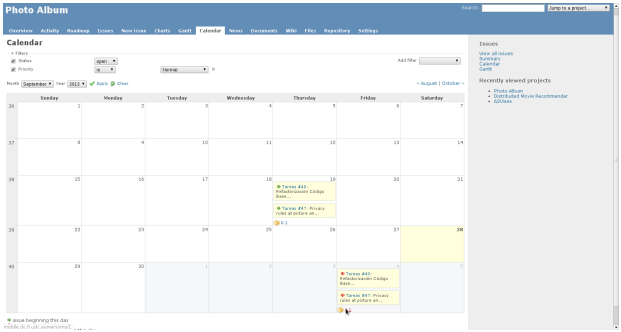
# The same issues, seen four ways

The **summary**: open and closed by type, priority, assignee, version, category.



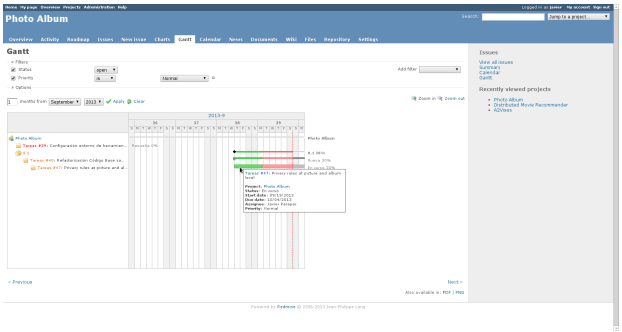
# The same issues, seen four ways

The **calendar**: what starts and what is due, this month.



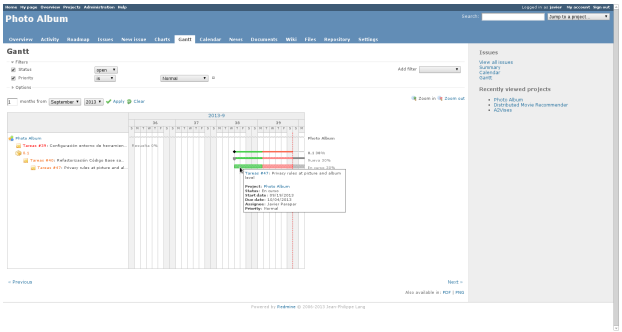
# The same issues, seen four ways

The **Gantt**: the same issues against the versions and their dates.



# The same issues, seen four ways

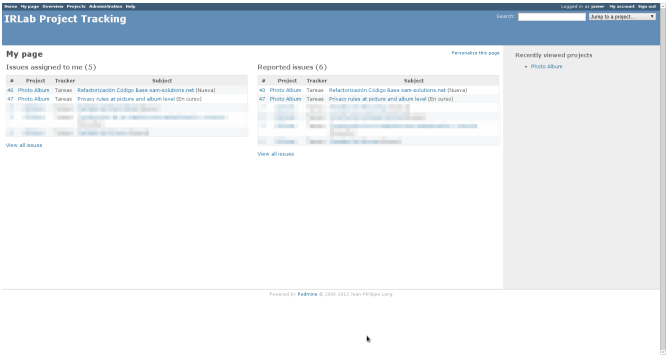
The **Gantt**: the same issues against the versions and their dates.




***Same data, four questions: what is there, how is it distributed, what is due now, will we make it. The Gantt only tells the truth if the dates and the versions are kept honest.***

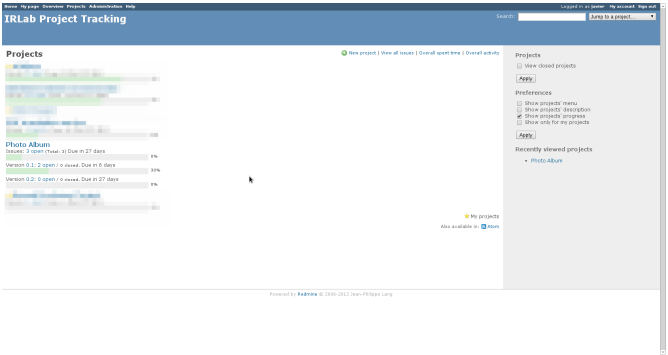
# And the view from outside the project

**My page**: what is assigned to me and what I reported, across every project.



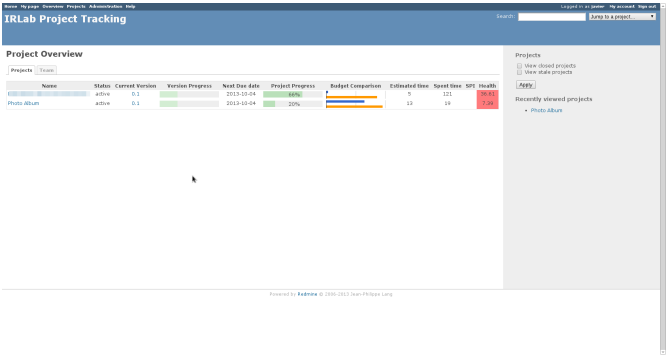
# And the view from outside the project

The **projects** list, each with its versions and its progress.



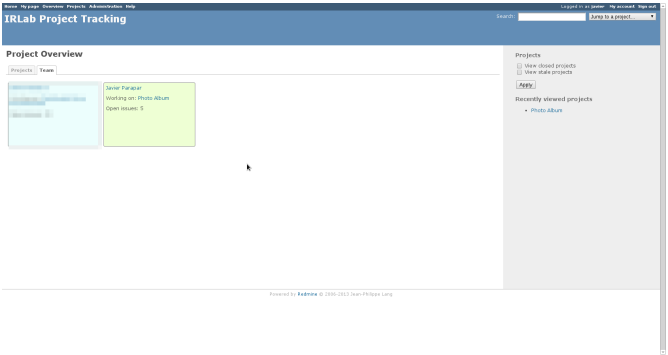
# And the view from outside the project

**Overview**: status, current version, progress, estimated against spent.



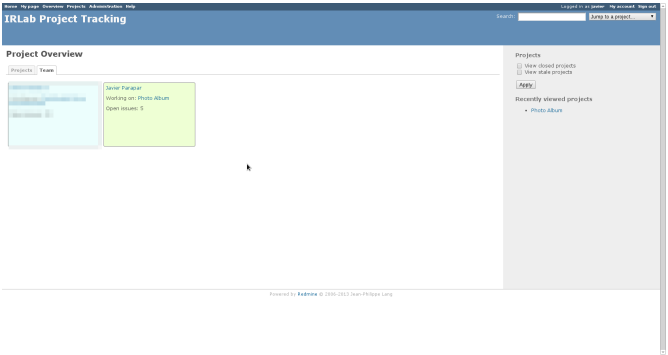
# And the view from outside the project

And the **team** view: who is working on what, right now.



# And the view from outside the project

And the **team** view: who is working on what, right now.




***My page is the one you should open every morning. The other three are the ones we open.***

SECTION 6

# Documents

6

# Not everything is an issue

A tracker holds **work**. What is not work has its own place, and putting each thing where it belongs is most of what “a well organised project” means:

| PLACE | FOR | LIFETIME |
|---|---|---|
| **Issue** | something somebody has to do | until it is closed |
| **Wiki** | how it works, why it was decided so | edited forever |
| **News** | an announcement to the whole team | read once |
| **Forum** | a discussion with no owner yet | until it becomes an issue |
| **Documents** | deliverables: specs, reports, manuals | versioned by hand |
| **Files** | binaries attached to a version | one per release |
| **Repository** | everything that is source | forever, with history |

# Not everything is an issue

A tracker holds **work**. What is not work has its own place, and putting each thing where it belongs is most of what "a well organised project" means:

| PLACE | FOR | LIFETIME |
|---|---|---|
| **Issue** | something somebody has to do | until it is closed |
| **Wiki** | how it works, why it was decided so | edited forever |
| **News** | an announcement to the whole team | read once |
| **Forum** | a discussion with no owner yet | until it becomes an issue |
| **Documents** | deliverables: specs, reports, manuals | versioned by hand |
| **Files** | binaries attached to a version | one per release |
| **Repository** | everything that is source | forever, with history |

The most common mistake is the opposite of what you would expect: teams put **everything** in issues, including the design of the system, and then nobody can find it because it is closed.

# Not everything is an issue

A tracker holds **work**. What is not work has its own place, and putting each thing where it belongs is most of what "a well organised project" means:

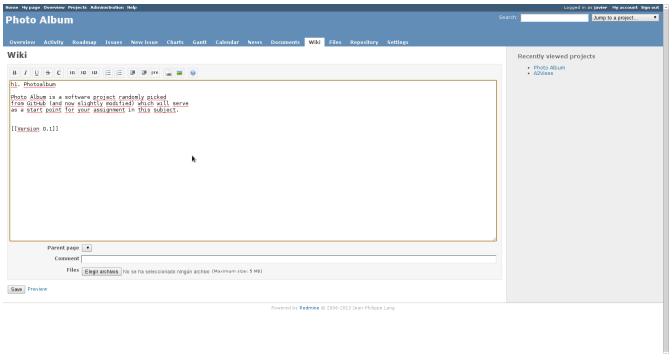
| PLACE | FOR | LIFETIME |
|---|---|---|
| **Issue** | something somebody has to do | until it is closed |
| **Wiki** | how it works, why it was decided so | edited forever |
| **News** | an announcement to the whole team | read once |
| **Forum** | a discussion with no owner yet | until it becomes an issue |
| **Documents** | deliverables: specs, reports, manuals | versioned by hand |
| **Files** | binaries attached to a version | one per release |
| **Repository** | everything that is source | forever, with history |

The most common mistake is the opposite of what you would expect: teams put **everything** in issues, including the design of the system, and then nobody can find it because it is closed.

***Rule of thumb: if it will still be true when the task is finished, it belongs in the wiki, not in the issue.***

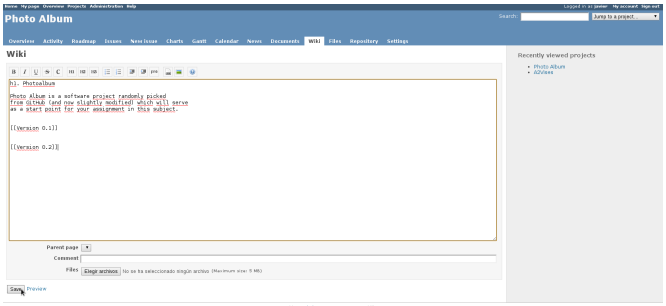
# The wiki

A page is plain text with light markup — headings, lists, code, links.



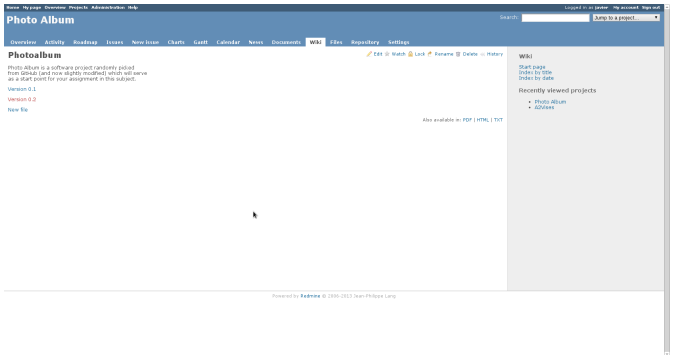
# The wiki

`[[Version 0.1]]` links to another page; write the link before the page exists.



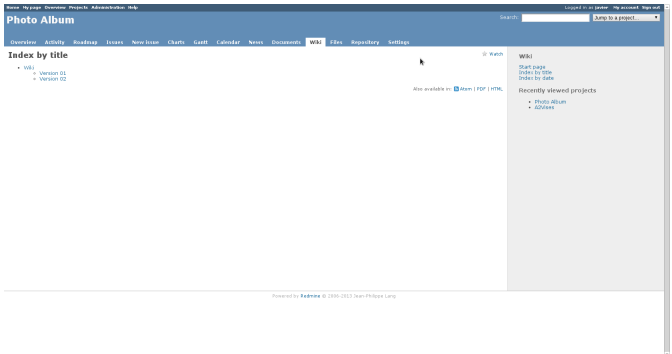
# The wiki

The result: a document the whole team edits, with history and diffs.



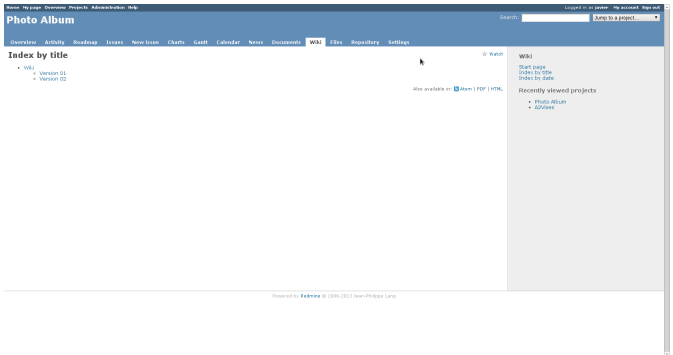
# The wiki

And an **index** that grows by itself, which is why the titles matter.



# The wiki

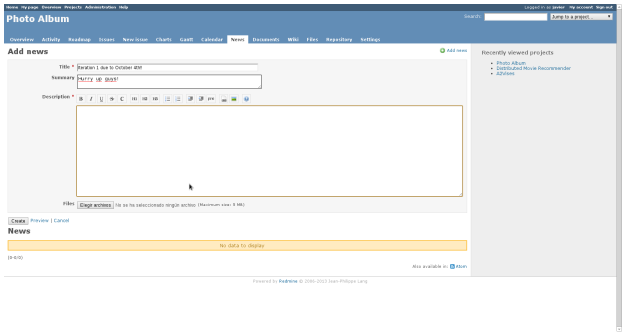
And an **index** that grows by itself, which is why the titles matter.




***Every page has a history, so the wiki is version controlled too — you can see who changed a decision, and when.***

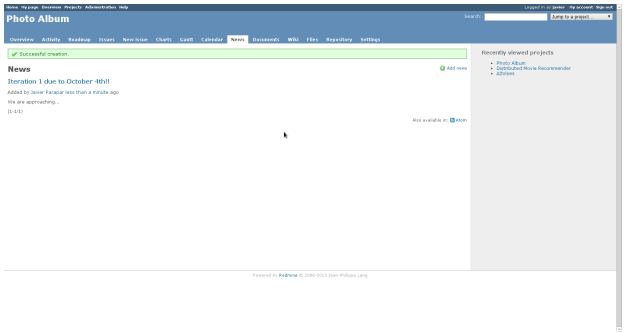
# News, documents and files

**News**: a dated announcement — a deadline, a decision, a broken build.



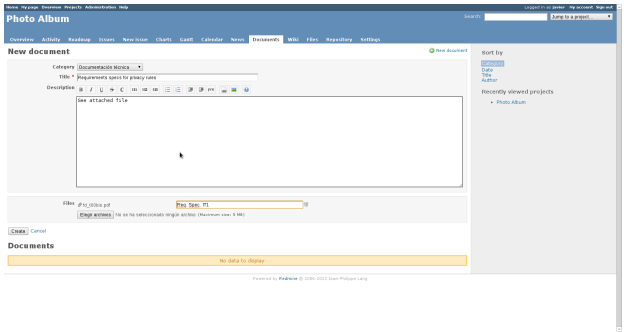
# News, documents and files

It appears on the project's front page and reaches the team by email.



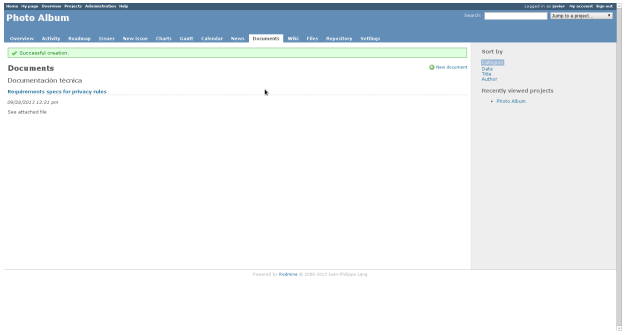
# News, documents and files

**Documents**: categorised, with a description and an attachment.



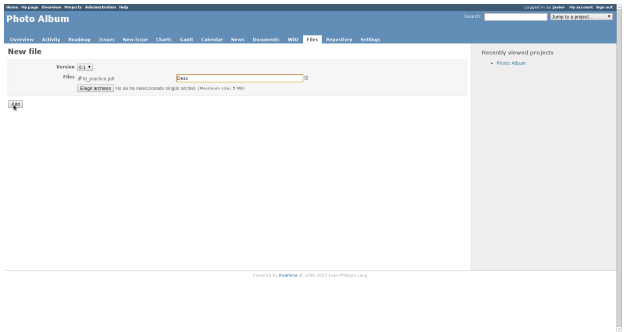
# News, documents and files

Which is where a deliverable that is not source code belongs.



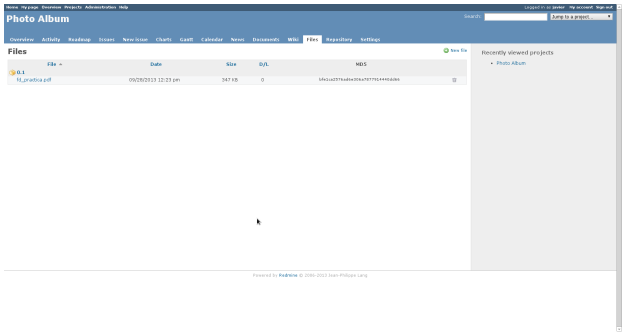
# News, documents and files

**Files**: attached **to a version** — a built artefact, a release.



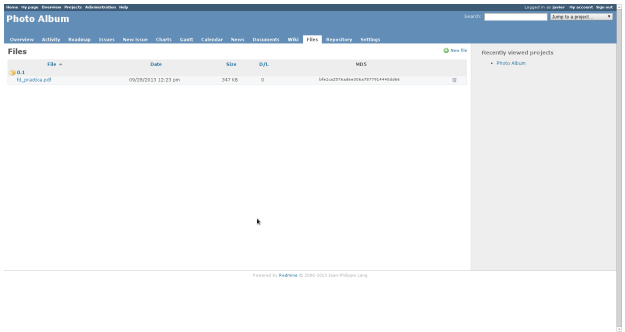
# News, documents and files

With size and checksum, so what you downloaded is what was published.



# News, documents and files

With size and checksum, so what you downloaded is what was published.




***Files is the manual version of something Lesson 5 will automate: from then on, the build publishes the artefact and nobody uploads anything by hand.***

SECTION 7

# Code

7

# The two words that join the tracker to the code

```
git commit -m "feat(trips): share a trip with a friend

A trip can now be shared with another user.  refs #42"

git commit -m "fix(login): stop the session expiring after one minute

closes #57 @1h30m"
```

`refs #42` **links** the commit to the issue. `closes #57` links it **and closes the issue** when the commit reaches the repository. `@1h30m` logs the time at the same moment. Redmine reads them when it fetches the repository; the issue then shows the commit, and the commit shows the issue.

# The two words that join the tracker to the code

```
git commit -m "feat(trips): share a trip with a friend

A trip can now be shared with another user.  refs #42"

git commit -m "fix(login): stop the session expiring after one minute

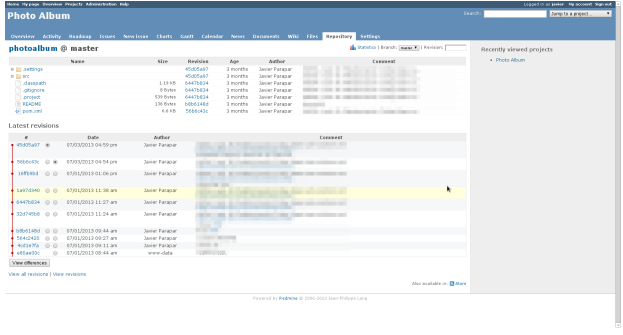
closes #57 @1h30m"
```

`refs #42` **links** the commit to the issue. `closes #57` links it **and closes the issue** when the commit reaches the repository. `@1h30m` logs the time at the same moment. Redmine reads them when it fetches the repository; the issue then shows the commit, and the commit shows the issue.

***This is the second half of the chain from the start of the lesson, and it is free: you were going to write a commit message anyway. Four extra characters make the history navigable in both directions, forever.***

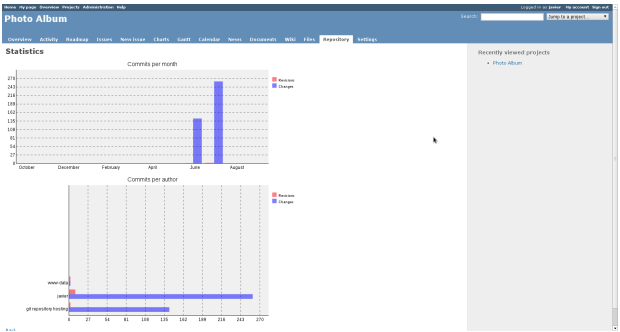
# The repository, from inside the tracker

The tree, the last revision of every file, and who touched it last.



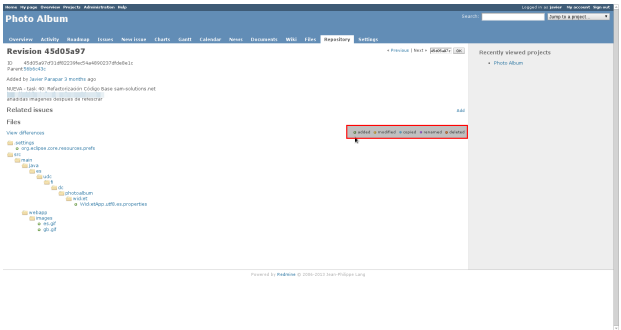
# The repository, from inside the tracker

**Statistics**: commits per month and per author — a rhythm, or the lack of one.



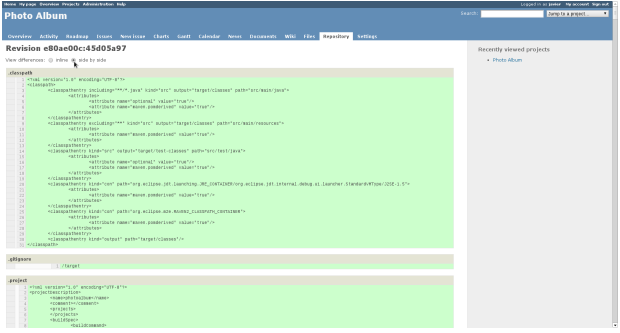
# The repository, from inside the tracker

A revision: its message, the files it touched, and the **issues it refers to**.



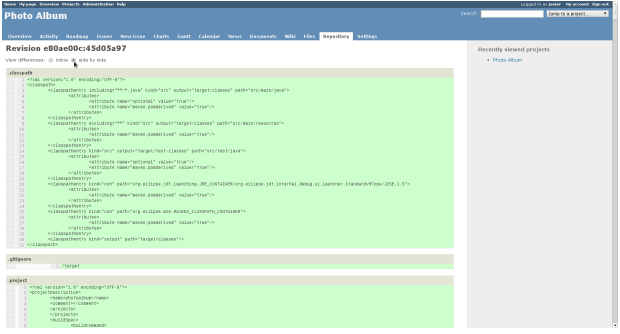
# The repository, from inside the tracker

And the diff, which is the same thing you read in a merge request.



# The repository, from inside the tracker

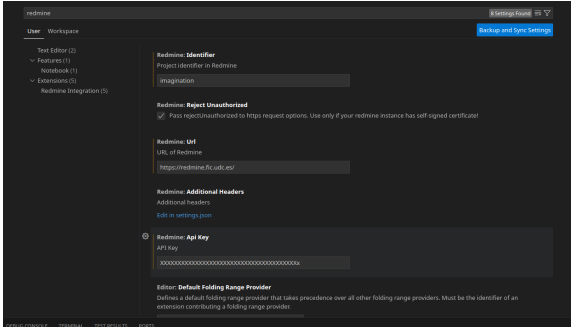
And the diff, which is the same thing you read in a merge request.




***The commits-per-author chart is not a ranking. It is a conversation starter, and the conversation is about how the team split the work — not about who types more.***

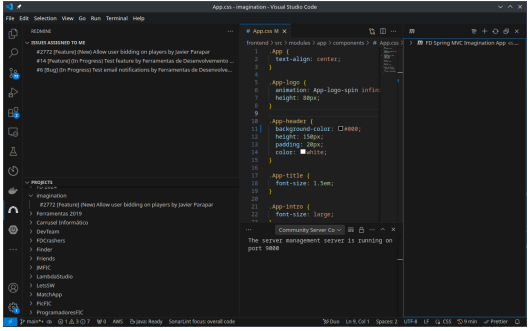
# The tracker where you are already working

The extension needs three things: the **URL**, your **API key** and the project.



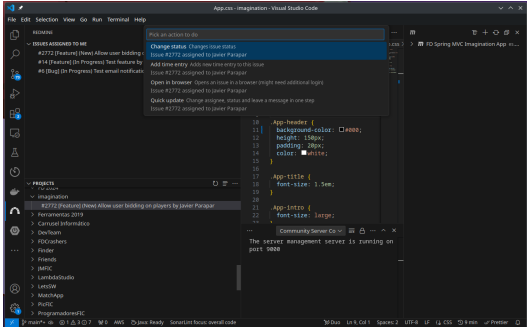
# The tracker where you are already working

And the issues assigned to you appear in the sidebar of the editor.



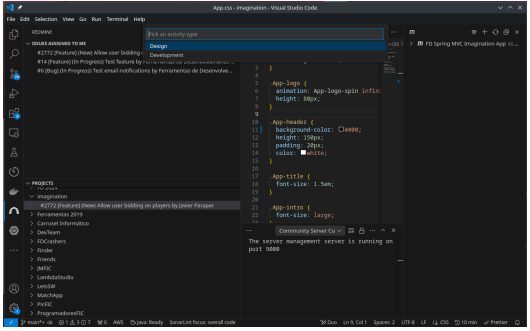
# The tracker where you are already working

From there: change status, log time, open in the browser, quick update.



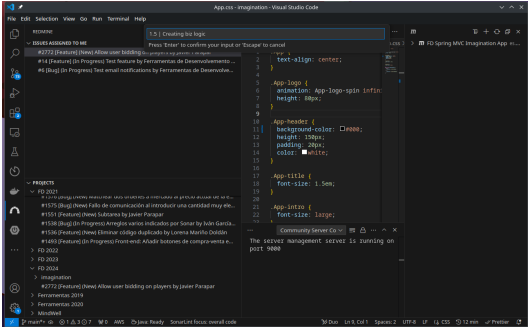
# The tracker where you are already working

The activity, without leaving the file you were editing…



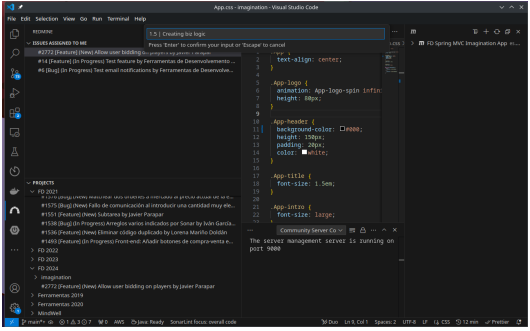
# The tracker where you are already working

… and the hours and the comment. Four keystrokes instead of a browser tab.



# The tracker where you are already working

… and the hours and the comment. Four keystrokes instead of a browser tab.




***This is the second principle of the lesson made practical: the information has to appear where the decision is taken. Time you log without changing window is time that actually gets logged.***

SECTION 8

# Working this way



# The rules, for the assignment

**Before the code**

- **The issue comes first**. No branch without an issue, no issue written after the work is done
- Every issue has a **tracker**, an **assignee** — one person — and a **target version**
- Issues are cut so that **one issue is one branch** of at most two days, as agreed in *Lesson 2*
- Bigger than that: a **parent task** with subtasks

**While you work**

- Move the **status** when the state changes, not at the end of the week
- `refs #42` in every commit; `closes #42` in the one that finishes it
- **Log your hours the same day**, with an activity and a comment
- Each iteration closes with its **version complete** and its **tag pushed**
- Decisions and designs go to the **wiki**, not into a closed issue

# The rules, for the assignment

**Before the code**

- **The issue comes first**. No branch without an issue, no issue written after the work is done
- Every issue has a **tracker**, an **assignee** — one person — and a **target version**
- Issues are cut so that **one issue is one branch** of at most two days, as agreed in *Lesson 2*
- Bigger than that: a **parent task** with subtasks

**While you work**

- Move the **status** when the state changes, not at the end of the week
- `refs #42` in every commit; `closes #42` in the one that finishes it
- **Log your hours the same day**, with an activity and a comment
- Each iteration closes with its **version complete** and its **tag pushed**
- Decisions and designs go to the **wiki**, not into a closed issue

***All of it is read when marking: the issues, their dates, their authors, the hours, and whether the repository agrees with the tracker. A project whose tracker was filled in the week before the deadline is visible from the first screen.***

# Your turn

1. Open your team's project in `redmine.fic.udc.es` and create the **four versions** of the course, with their dates.
2. Agree **four or five categories** for the parts of your system, and a default assignee for each.
3. Write the **first five issues** of iteration 1: one of them a parent task with two subtasks, all of them with a target version.
4. Take one of them, create its branch as in *Lesson 2*, and make one commit with `refs #N` in the message. Check that the issue shows it.
5. Log the time it took you, with an activity, **today**.
6. Write one wiki page: how to build and run your project from scratch.

# Your turn

1. Open your team's project in `redmine.fic.udc.es` and create the **four versions** of the course, with their dates.
2. Agree **four or five categories** for the parts of your system, and a default assignee for each.
3. Write the **first five issues** of iteration 1: one of them a parent task with two subtasks, all of them with a target version.
4. Take one of them, create its branch as in *Lesson 2*, and make one commit with `refs #N` in the message. Check that the issue shows it.
5. Log the time it took you, with an activity, **today**.
6. Write one wiki page: how to build and run your project from scratch.

The sixth one is the one you will thank yourselves for in November, when somebody reinstalls their machine two days before a deadline.

# What to take away

**The tracker is the shared memory of the team.** Chat forgets, email forgets, and six people remember six different projects.

**The issue is the unit of work**, and its number is what holds the chain together: issue, branch, merge request, commit, version, tag.

**An issue is written for somebody else.** A symptom, steps, and how we will know it is done — or it is a note, not an issue.

**Statuses and hours are only worth what they cost you.** Moved the same day they change, they are a plan; filled in at the end, they are fiction.

**And everything that is not work has its own place**: the wiki for what stays true, the repository for what is source.

# What to take away

**The tracker is the shared memory of the team.** Chat forgets, email forgets, and six people remember six different projects.

**The issue is the unit of work**, and its number is what holds the chain together: issue, branch, merge request, commit, version, tag.

**An issue is written for somebody else.** A symptom, steps, and how we will know it is done — or it is a note, not an issue.

**Statuses and hours are only worth what they cost you.** Moved the same day they change, they are a plan; filled in at the end, they are fiction.

**And everything that is not work has its own place**: the wiki for what stays true, the repository for what is source.

Next: *Lesson 4* — building the project, always the same way, on anybody's machine.

# Appendix: two templates you can copy

## Bug

*Subject*: symptom, one line

**Steps**

1. …  2. …  3. …

**Expected**: …
**Actual**: …

**Environment**: branch, version, browser
**Evidence**: trace / screenshot / log

## Task

*Subject*: the outcome, one line

**Context**: why this is needed

**Scope**: what it must do
**Out of scope**: what it must not

**Done when**:

— …
— …

Paste them into the description field. Two minutes of writing saves the half hour somebody else would spend asking.

# Appendix: the defaults, and where to change them

| THING | OUT OF THE BOX | CHANGED IN |
|---|---|---|
| Trackers | Bug, Feature, Support | Administration |
| Statuses | New, In Progress, Resolved, Feedback, Closed, Rejected | Administration |
| Priorities | Low, Normal, High, Urgent, Immediate | Administration |
| Activities | design, development, testing… | Administration / project |
| Workflow | which role may make which transition | Administration |
| Categories | none — you define them | Project settings |
| Versions | none — you define them | Project settings |
| Members and roles | — | Project settings |
| Commit keywords | `refs`, `references` / `fixes`, `closes` | Administration |

The first column is what you will meet in your project; the third is why some of it looks different in another installation. The workflow — who may move what, and to where — is the one that surprises people: it depends on the role and on the tracker.